MODULE 2

Syllabus

The Data Encryption Standard - DES Encryption & Decryption, Avalanche Effect, Strength of DES; Advanced Encryption Standard - AES Structure; Stream Ciphers; RC4; Principles of Public-Key Cryptosystems - Public- Key Cryptosystems, Applications for Public-Key Cryptosystems, Requirements for Public-Key Cryptography.

Data Encryption Standard (DES)

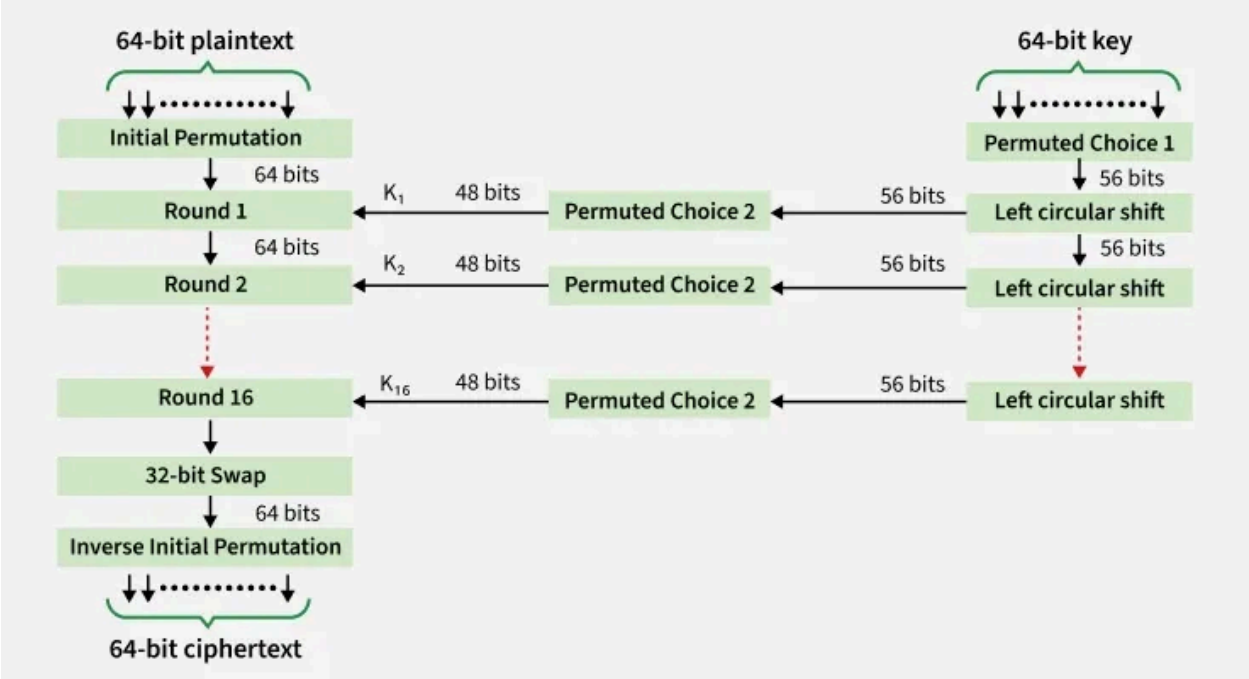
Data Encryption Standard (DES) is a symmetric block cipher. By 'symmetric', we mean that the size of input text and output text (ciphertext) is same (64-bits). The 'block' here means that it takes group of bits together as input instead of encrypting the text bit by bit. Data encryption standard (DES) has been found vulnerable to very powerful attacks and therefore, it was replaced by Advanced Encryption Standard (AES).

- It is a block cipher that encrypts data in 64 bit blocks.
- It takes a 64-bit plaintext input and generates a corresponding 64-bit ciphertext output.
- The main key length is 64-bit which is transformed into 56-bits by skipping every 8th bit in the key.
- It encrypts the text in 16 rounds where each round uses 48-bit subkey.
- This 48-bit subkey is generated from the 56-bit effective key.

- The same algorithm and key are used for both encryption and decryption with minor changes.

Working of Data Encryption Standard (DES)

DES is based on the two attributes of Feistel cipher i.e. Substitution (also called confusion) and Transposition (also called diffusion). DES consists of 16 steps, each of which is called a round. Each round performs the steps of substitution and transposition along with other operations.



The encryption starts with a 64-bit plaintext that needs to be encrypted using a 64-bit key. Plaintext is passed to Initial Permutation function and key is permuted using Permuted Choice 1 (PC-1).

Initial Permutation

The 64-bit plaintext block is input into an Initial Permutation (IP) function that rearranges the order of bits. The order of bits is changed using

predefined table. The IP table is a 8×8 matrix (64 entries) where each entry specifies the new position of a bit from the original plaintext.

Initial Permutation Table							
58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

Working of IP Table:

- The first bit of the permuted block is taken from the 58th bit of the original plaintext.
- The second bit comes from the 50th bit and so on.
- The last (64th) bit comes from the 7th bit of the original plaintext.

The initial permutation (IP) happens only once and it happens before the first round. The permutation this function do is fixed and does not depend on the plaintext. This rearranged 64-bit plaintext then go through 16 rounds. Each of this round uses a different 48-bit subkey from the previous round subkey. These subkeys are generated from 64-bit key.

Key Transformation

The 64-bit initial key is converted into 56-bit effective key. This 56-bit key further generates 48-bit subkeys for each of the 16 Feistel rounds.

Conversion of 64-bit Key into 56-bit Key

Initial key first go through Permuted Choice 1 (PC-1) which reduces the key to 56 bits. In PC-1 every eighth bit in key is discarded. That is bit positions 8, 16, 24, 32, 40, 48, 56, and 64 are discarded.

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32
33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48
49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64

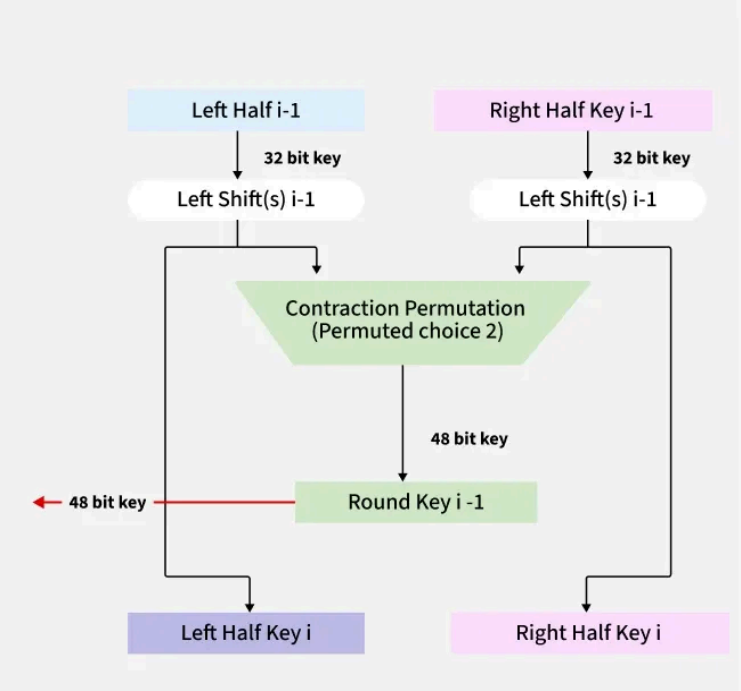
These discarded bits are called parity bits which are used for error checking. Remaining 56 bits are split into two 28-bit halves:

- Left Half (C_i): First 28 bits.
- Right Half (D_i): Last 28 bits

Here i represent the number of the Feistel round.

Generating 48-bit Round Subkeys

For each of the 16 rounds, right half (C_i) and left half (D_i) undergo circular left shift operation.



For Feistel round 1, 2, 9, and 16 both halves (left and right) undergo 1-bit left shift operation. For others rounds (3, 4, 5, 6, 7, 8, 10, 11, 12, 13, 14, 15) the halves undergo 2-bit left shift operation.

Round	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
#key bits shifted	1	1	2	2	2	2	2	2	1	2	2	2	2	2	2	1

After circular shift operation is performed, C_i and D_i are again combined into 56-bit block. This block then go through Permutation Choice 2 (PC-2). The PC-2 selects and arrange 48 bits out of the 56 to form the round subkey (K_i). These 48 bits are selected on the basis of a predefined table as shown below:

Permutation Choice 2 Table							
14	17	11	24	1	5	3	28
15	6	21	10	23	19	12	4
26	8	16	7	27	20	13	2
41	52	31	37	47	55	30	40
51	45	33	48	44	49	39	56
34	53	46	42	50	36	29	32

According to this table 14th bit is placed to first position, 17th bit to second position, 11th bit to 3rd position and so on. The output 48-bit subkey of this table is used to cipher the plaintext in the Feistel round.

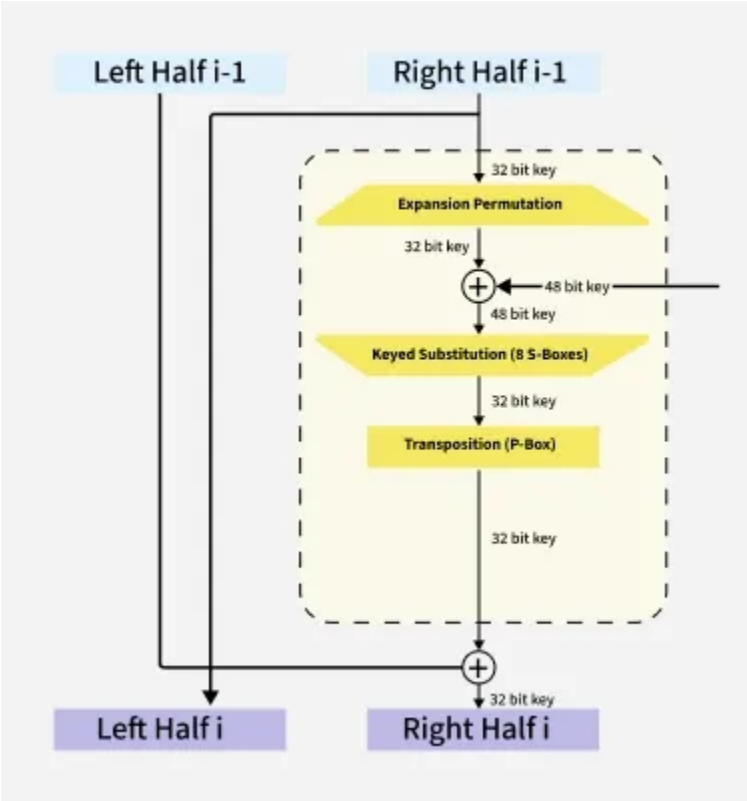
For next round we use already left shifted C_i and D_i as left and right half. We again perform the circular left shift operation on both halves. We again combine the result into 56-bit block and use permutation choice 2 to contract this block into 48-bit subkey for next round.

The process of Circular Left Shift and Permutation Choice 2 is followed for 16 rounds and different round subkey (K_i) is generated for each Feistel Round.

Each 48-bit subkey (K_i) is XORed with the expanded right half in the Feistel Round. Below is the explanation of what happens in every single Feistel round.

Feistel Rounds (1 - 16)

Every round receives 64-bits permuted plaintext from the Initial Permutation function and 48-bit transformed subkey (K_i). The permuted 64-bit plaintext is divided into two halves called as Left Plaintext (LPT) and Right Plaintext (RPT). Both of these halves are 32 bit in size. The right half or Right Plaintext (RPT) is processed using Mangler (F) function. Mangler (F) function involves expansion, key mixing, substitution (S-boxes), and permutation (P-box) of RPT.



The RPT first go through Expansion Permutation. In this permutation 32-bit Right Plaintext (RPT) is expanded into 48 bits using expansion box or E-box table.

E-Box Expansion Table					
32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

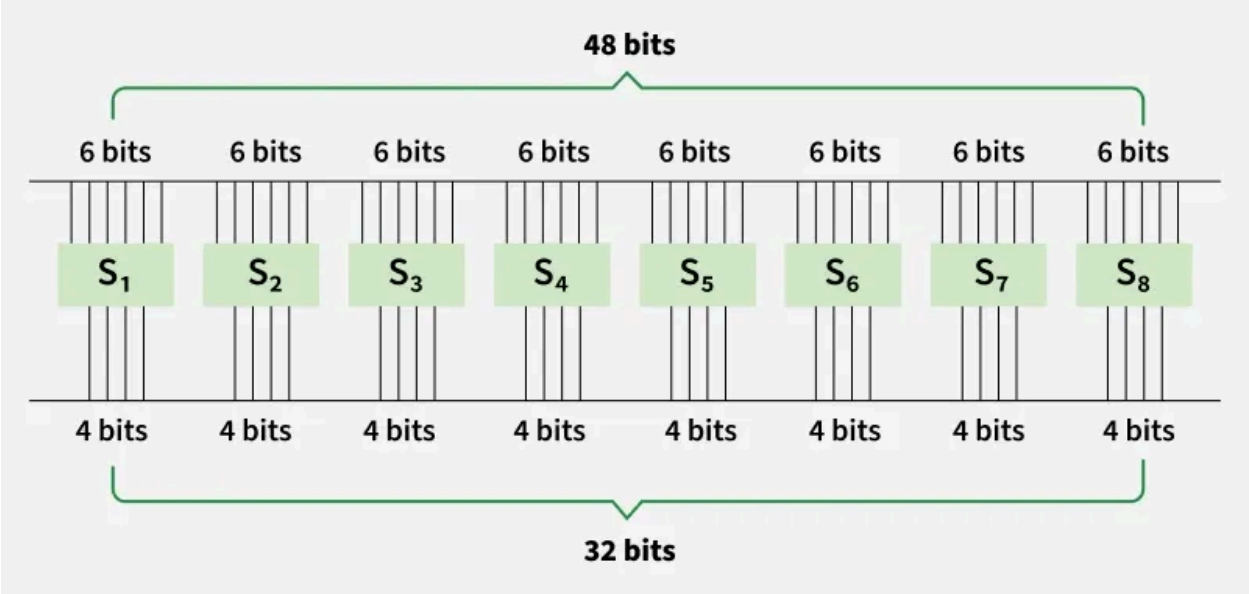
The 48-bit expanded block is generated by arranging the bits as in E-Box table.

This expanded block is XORed (⊕) with the 48-bit round subkey that we generated during key transformation process. The XOR or Exclusive OR operation returns '0' as output if both inputs are same, else the out will be '1'. After XOR is performed, the resulting 48-bit block is split into eight chunks of 6-bit size each. Each of the chunk is then fed into a different S-box (S_1 to S_8).

For example, the output of XOR operation is converted into 6 bit chunks as follows:

101010 010001 011110 111010 100001 100110 010100 100111

These 6 bits chunks will be converted into 4 bits using S-Boxes.



S-Boxes are predefined lookup tables which reduces 6 bits chunk into 4 bits. Below is the list of these S-Boxes.

SBox-8

	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
00	13	2	8	4	6	15	11	1	10	9	3	14	5	0	12	7
01	1	15	13	8	10	3	7	4	12	5	6	11	0	14	9	2
10	7	11	4	1	9	12	14	2	0	6	10	13	15	3	5	8
11	2	1	14	7	4	10	8	13	15	12	9	0	3	5	6	11

Suppose the first 6-bit chunk is 101010. We divide this chunk into two parts of 2 bit and 4 bit size. First and last bits are combined together for 2-bit part and rest bits make up the 4-bit part.

101010 -> (1)(0101)(0) -> divided into 10 and 0101

We look for these parts in the rows and columns of S_1 table. The number in the cell where row is '10' and column is '0101' is '6' in the S_1 table. The binary value of six is '0110'. This is the 4 bit value that S-Box 1 generated from the 6 bit input '101010'.

6-bit chunk: '101010' converted into 4-bit chunk: '0110'

Similarly we convert every 6-bit chunk into 4-bit value using S-Boxes. This process is called substitution. After that we combine all of these 4-bit chunks to get 32-bit block as output. This 32 bit again get permuted using following table.

P-box Permutation							
16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

This permutation is called Transposition. The mangler function finishes here. 32-bit block after permutation is the output of mangler function. This block is

XORed with 32-bit Left half or Left Plaintext (LPT) that was generated in the beginning of the Feistel round after Initial Permutation (IP). The output of this XOR operation serves as Right Half or Right Plaintext for next round and the initial Right Half (RPT) will serve as Left Half for the next round.

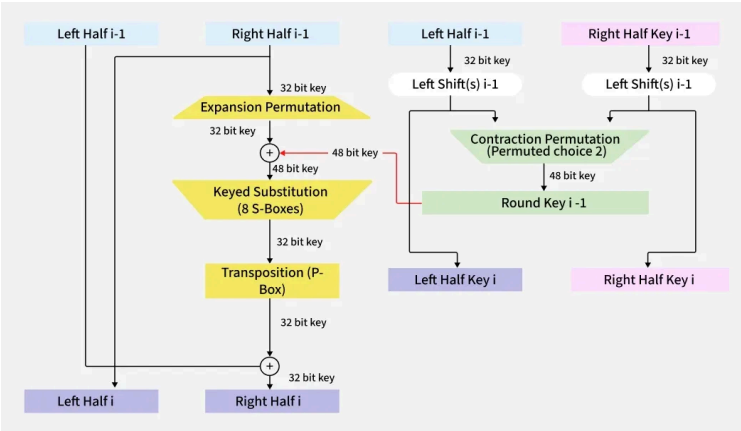
$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus F(R_{i-1}, K_i)$$

Where

- L_{i-1} = The Left Half or Left Plaintext (LPT) of current round.
- L_i = The Left Half or Left Plaintext (LPT) for next round.
- R_{i-1} = The Right Half or Right Plaintext (LPT) of current round.
- R_i = The Right Half or Right Plaintext (LPT) for next round.

We do the same operations as mentioned for 16 rounds using subkeys generated by key transformation. The whole process is shown below in the diagram.



32-bit Swap and Inverse Initial Permutation

After these 16 rounds we get two blocks (Left and Right) of 32-bit each. The two 32-bit halves are again swapped back, resulting in a 64-bit block. This step is called 32-bit Swap in DES encryption algorithm.

Finally, the block undergoes an Inverse Initial Permutation (IP-1). This is essentially the inverse of the initial permutation applied at the beginning.

Inverse Initial Permutation							
Output Position	Input Position	Output Position	Input Position	Output Position	Input Position	Output Position	Input Position
58	1	**62**	17	**57**	33	**61**	49
50	2	**54**	18	**49**	34	**53**	50
42	3	**46**	19	**41**	35	**45**	51
34	4	**38**	20	**33**	36	**37**	52
26	5	**30**	21	**25**	37	**29**	53
18	6	**22**	22	**17**	38	**21**	54
10	7	**14**	23	**9**	39	**13**	55
2	8	**6**	24	**1**	40	**5**	56
60	9	**64**	25	**59**	41	**63**	57
52	10	**56**	26	**51**	42	**55**	58
44	11	**48**	27	**43**	43	**47**	59
36	12	**40**	28	**35**	44	**39**	60
28	13	**32**	29	**27**	45	**31**	61
20	14	**24**	30	**19**	46	**23**	62
12	15	**16**	31	**11**	47	**15**	63
4	16	**8**	32	**3**	48	**7**	64

The result of the inverse initial permutation is the final 64-bit ciphertext which is the encrypted version of the original plaintext.

Decryption in DES (Data Encryption Standard)

Decryption in DES follows the same process as encryption but in reverse order. Since DES is a symmetric-key algorithm, the same key is used for both encryption and decryption, but the subkeys (round keys) are applied in reverse order.

- **Reverse Subkey Application**: The 16 round keys generated during key scheduling are used in reverse order (from K_{16} to K_1) during decryption.
- **Inverse Feistel Function**: The Feistel network structure ensures that decryption mirrors encryption. Each round performs the same operations (expansion, S-box substitution, permutation), but with reversed subkeys.
- **Final Permutation (FP)**: After 16 rounds, the output undergoes the Inverse Initial Permutation (IP), reversing the initial shuffling.

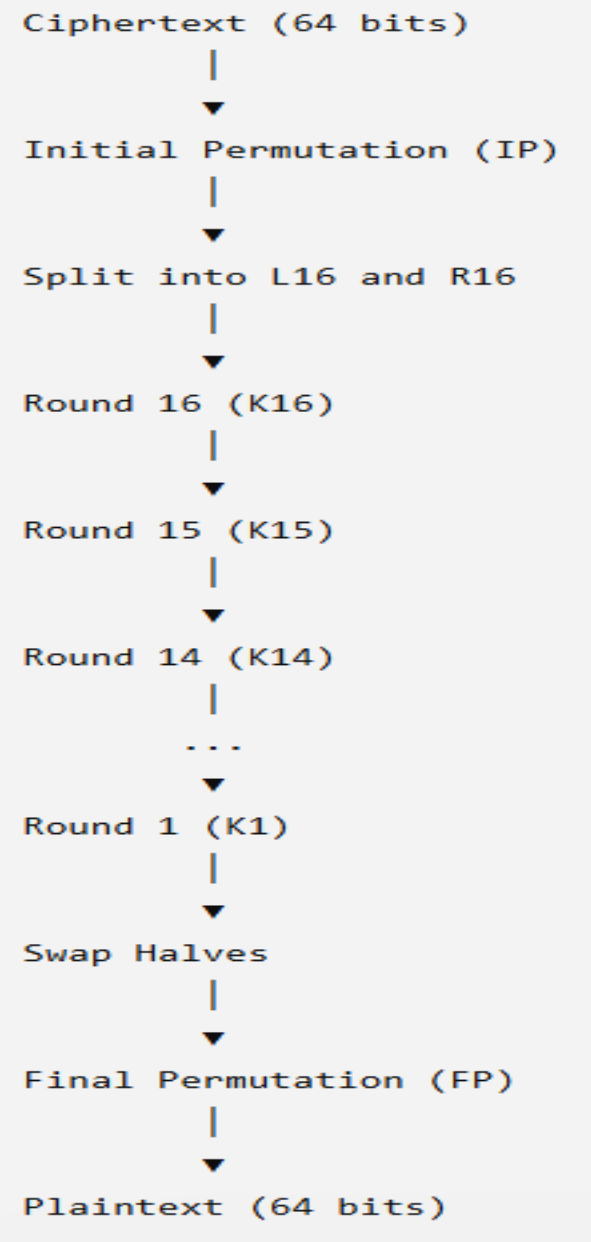
DES Decryption Process

The decryption process follows these steps:

1. Receive the ciphertext.
2. Apply the Initial Permutation (IP).
3. Split the data into left and right halves.
4. Perform the same 16 Feistel rounds.
5. Use the round keys in reverse order (**K16 to K1**).

6. Swap the halves.
7. Apply the Final Permutation (FP).
8. Obtain the original plaintext.

DES Decryption Block Diagram



Why is the Same Algorithm Used for Decryption?

DES is based on the **Feistel Network**.

A Feistel network has a unique property:

> The encryption process can be reversed simply by applying the round keys in the reverse order.

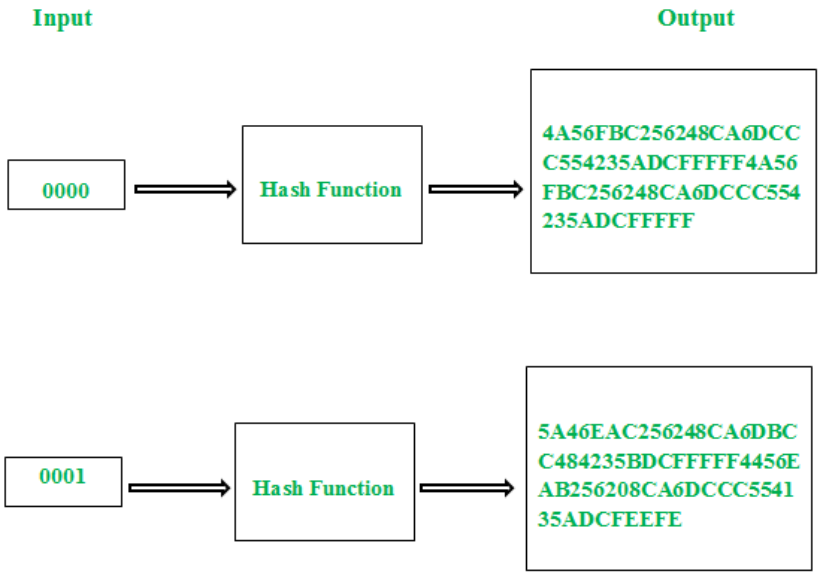
Therefore:

- No separate decryption algorithm is required.
- Only the order of the keys changes.

This reduces implementation complexity.

Avalanche Effect

The avalanche effect means that a small change in the input of a cryptographic system causes a big and unpredictable change in the output. For example, changing one bit in a message makes the whole encrypted message look very different. This helps keep the data secure because it makes it hard for anyone to figure out the original message or key.



Even though the concept of avalanche effect was identified by "Shannon's property of confusion", the term was first mentioned by Horst Feistel. To implement a strong cipher or cryptographic hash function, this should be considered as one of the primary design objective.

In case of algorithm that uses hash value, even a small alteration in an input string should drastically change the hash value. In other words, flipping single bit in input string should at least flip half of the bits in the hash value.

Avalanche Effect examples

Here are 5 examples demonstrating the avalanche effect with a simple change in the message:

AES Encryption

- **Message**: "Hello World"
- **Ciphertext**: "f7e63cf44dd4e4e8a511f688c8ab5786"

- **Changed Message**: "Hella World"
- **New Ciphertext**: "0fbfa517a5e8a6dbd87a1265e41e567c"

SHA-256 Hash Function

- **Message**: "Hello World"
- **Hash**: "a830d7beb04eb7549ce990fb7dc962e499a27230c6a5baef2e4e6b91132f0899"
- **Changed Message**: "Hello Wxrld"
- **New Hash**: "d2a1f9351a469f2dcf8a91f5db81d9c63562eb9b3188ef0d9fbfae56232c9c94"

MD5 Hash Function

- **Message**: "Hello World"
- **Hash**: "b10a8db164e0754105b7a99be72e3fe5"
- **Changed Message**: "Hello Werld"
- **New Hash**: "88965b63db1b7d8c10e6dfd3d2a682a4"

Blowfish Encryption

- **Message**: "Hello World"

- **Ciphertext**: "328c57baf0c7b9d4e1f0aafde45d3ed4"
- **Changed Message**: "Hello Word"
- **New Ciphertext**: "7b4edc2f1a0549a6b598f2b62b64e8f5"

DES Encryption

- **Message**: "Hello World"
- **Ciphertext**: "4a3c5a2244d2d7aa"
- **Changed Message**: "Hollo World"
- **New Ciphertext**: "9b2c4f5c6381e3cc"

The Criteria For the Avalanche Effect

In cryptography, the criteria for evaluating the quality of the avalanche effect in block ciphers include the **Strict Avalanche Criterion (SAC)** and the **Bit Independence Criterion (BIC)**:

Strict Avalanche Criterion (SAC)

1. The SAC requires that if a single bit in the input of a cryptographic function is flipped, each output bit should change with a probability of 50%.
2. Ensures that small changes in the input result in significant and widespread changes in the output, enhancing unpredictability.
3. **Example**: If you change one bit in the input, about half of the bits in the output should flip from their original state.

Bit Independence Criterion (BIC)

1. The BIC ensures that changing one bit in the input affects the output bits independently of each other.
2. Guarantees that no correlation exists between the changes in output bits, ensuring high randomness and complexity in the output.
3. **Example**: If two output bits change due to a single input bit flip, these changes should be independent of each other, not following a predictable pattern.

A good encryption algorithm should always satisfy the following relation:

`Avalanche effect > 50%`

The effect ensures that an attacker cannot easily predict a plain-text through a statistical analysis. An encryption algorithm that doesn't satisfies this property can favor an easy statistical analysis. That is, if the alteration in a single bit of the input results in change of only single bit of the desired output, then it's easy to crack the encrypted text.

Strengths of DES

DES has several important strengths that made it a reliable encryption algorithm for many years.

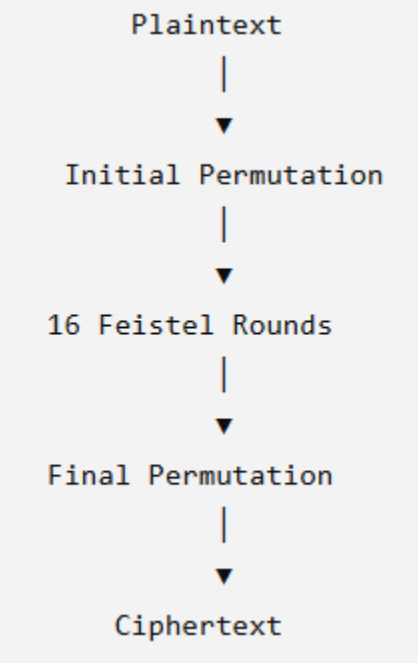
1. Strong Feistel Structure

DES is based on the **Feistel Network**, which divides the plaintext into two equal halves and processes them through **16 rounds** of encryption.

Advantages of the Feistel Structure

- Simple algorithm design.
- Same algorithm is used for both encryption and decryption.
- Efficient implementation in hardware.
- Easy to reverse during decryption by using the round keys in reverse order.

Diagram



The Feistel structure simplifies the implementation because encryption and decryption use the same operations.

2. Good Confusion and Diffusion

Claude Shannon proposed two important principles for secure encryption:

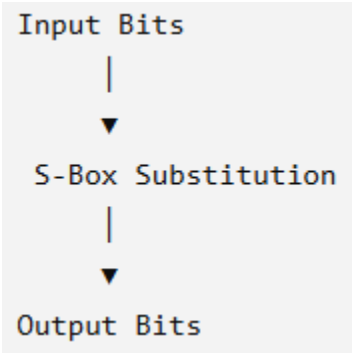
- **Confusion**
- **Diffusion**

DES successfully implements both.

Confusion

Confusion makes the relationship between the encryption key and the ciphertext highly complex.

DES achieves confusion using **S-Boxes (Substitution Boxes)**.



Diffusion

Diffusion spreads the influence of one plaintext bit over many ciphertext bits.

DES achieves diffusion using:

- Initial Permutation
- Expansion Permutation
- P-Box Permutation
- 16 rounds of processing

3. Avalanche Effect

A secure encryption algorithm should exhibit the **Avalanche Effect**.

Definition

A small change in the plaintext or key should produce a large change in the ciphertext.

Ideally,

Changing **one input bit** should change approximately **half of the output bits**.

Example

Plaintext

1010101010101010

↓

Ciphertext

1001010110111001

Change One Bit

1010101010101011

↓

Ciphertext

0110101001100100

The ciphertext changes completely even though only one input bit changed.

Importance

- Prevents attackers from identifying patterns.
- Improves resistance against cryptanalysis.

- Enhances security.

4. High Resistance to Differential Cryptanalysis

Differential cryptanalysis studies how differences in the plaintext affect differences in the ciphertext.

DES was specifically designed to resist this attack.

Reasons include:

- Carefully designed S-Boxes.
- Multiple rounds.
- Strong diffusion.

At the time DES was introduced, differential cryptanalysis was extremely difficult.

5. Efficient Hardware Implementation

DES performs very efficiently in hardware.

Applications include:

- ATM machines
- Smart cards
- Banking equipment
- Embedded systems

Hardware implementation provides:

- High speed
- Low processing delay
- Reliable operation

6. Standardized Algorithm

DES was officially standardized by the U.S. government.

Benefits include:

- International acceptance.
- Interoperability between systems.
- Easy implementation in commercial products.
- Extensive research and testing.

7. Large Number of Possible Keys

DES uses a **56-bit key**.

Total possible keys:

$$2^{56}$$

Approximately:

72,057,594,037,927,936

At the time DES was introduced, this key space was considered sufficiently large to prevent brute-force attacks.

8. Well-Tested Algorithm

DES remained in practical use for over **20 years**.

During this period:

- Millions of systems used DES.
- Many attacks were studied.
- Most weaknesses became well understood.

This extensive analysis helped in the development of more secure algorithms such as AES.

Weaknesses of DES

Although DES was highly secure in the 1970s, technological advancements exposed several weaknesses.

1. Small Key Size

The biggest weakness of DES is its **56-bit effective key**.

Although the original key contains **64 bits**, 8 bits are used for parity checking.

Therefore,

Original Key = 64 bits

↓

Effective Key = 56 bits

Modern computers can search all possible keys within a practical time.

2. Vulnerable to Brute-Force Attack

A **brute-force attack** attempts every possible key until the correct one is found.

Working

Ciphertext

↓

Try Key 1

↓

Wrong

↓

Try Key 2

↓

Wrong

↓

...

↓

Correct Key

↓

Plaintext

Since DES has only **2^56** possible keys, specialized hardware can successfully perform brute-force attacks.

3. Slow in Software

DES was primarily designed for hardware implementation.

Software implementations require:

- Many permutations
- Multiple substitutions
- Sixteen rounds

Consequently, DES is slower than modern algorithms like AES in software environments.

4. Susceptibility to Linear Cryptanalysis

Linear cryptanalysis attempts to establish linear relationships between the plaintext, ciphertext, and key.

Although DES offers some resistance, it is not completely immune.

Large amounts of known plaintext can help attackers perform successful linear cryptanalysis.

5. Short Block Size

DES encrypts only **64-bit blocks**.

Small block sizes increase the probability of repeated ciphertext patterns when encrypting large amounts of data.

Modern algorithms such as AES use **128-bit blocks**, providing better security.

6. Obsolete Security Standard

DES no longer provides sufficient security for modern applications.

Reasons include:

- Faster processors.
- Powerful GPUs.
- Cloud computing.
- Specialized cracking hardware.

Today, DES is considered obsolete.

7. Weak Keys

Certain DES keys produce undesirable encryption behavior.

Examples include:

- Weak Keys
- Semi-Weak Keys

These keys reduce the effectiveness of encryption and are generally avoided in practical implementations.

8. Replaced by Better Algorithms

To overcome the limitations of DES:

Triple DES (3DES)

Encryption is performed three times.

Plaintext

↓

Encrypt (K1)

↓

Decrypt (K2)

↓

Encrypt (K3)

↓

Ciphertext

This improves security but reduces performance.

Advanced Encryption Standard (AES)

AES provides:

- Larger key sizes (128, 192, and 256 bits)
- Faster execution
- Better resistance to attacks
- Higher efficiency in both hardware and software

Advanced Encryption Standard (AES)

Advanced Encryption Standard (AES) is a highly trusted encryption algorithm used to secure data by converting it into an unreadable format without the proper key. It is developed by the National Institute of Standards and Technology (NIST) in 2001. It is is widely used today as it is much stronger than DES and triple DES despite being harder to implement. AES encryption uses various key lengths (128, 192, or 256 bits) to provide strong protection against unauthorized access. This data security measure is efficient and widely implemented in securing internet communication, protecting sensitive data, and encrypting files. AES, a cornerstone of modern cryptography, is recognized globally for its ability to keep information safe from cyber threats.

- AES is a Block Cipher.
- The key size can be 128/192/256 bits.
- Encrypts data in blocks of 128 bits each.

That means it takes 128 bits as input and outputs 128 bits of encrypted cipher text. AES relies on the substitution-permutation network principle, which is performed using a series of linked operations that involve replacing and shuffling the input data.

Working of The Cipher

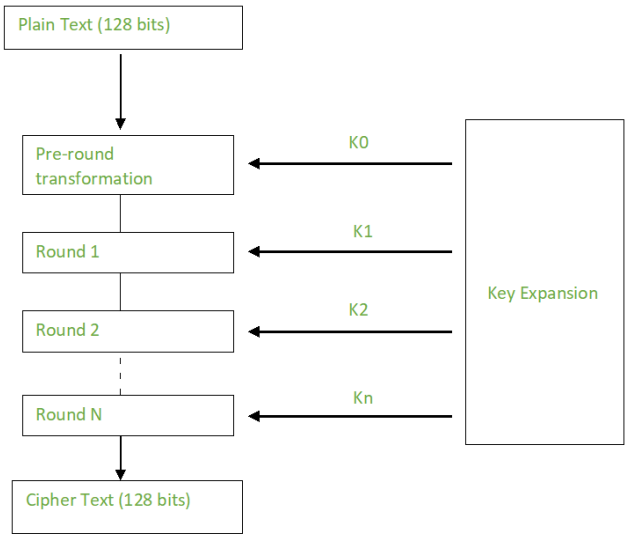
AES performs operations on bytes of data rather than in bits. Since the block size is 128 bits, the cipher processes 128 bits (or 16 bytes) of the input data at a time.

The number of rounds depends on the key length as follows :

N (Number of Rounds)	Key Size (in bits)
10	128
12	192
14	256

Creation of Round Keys

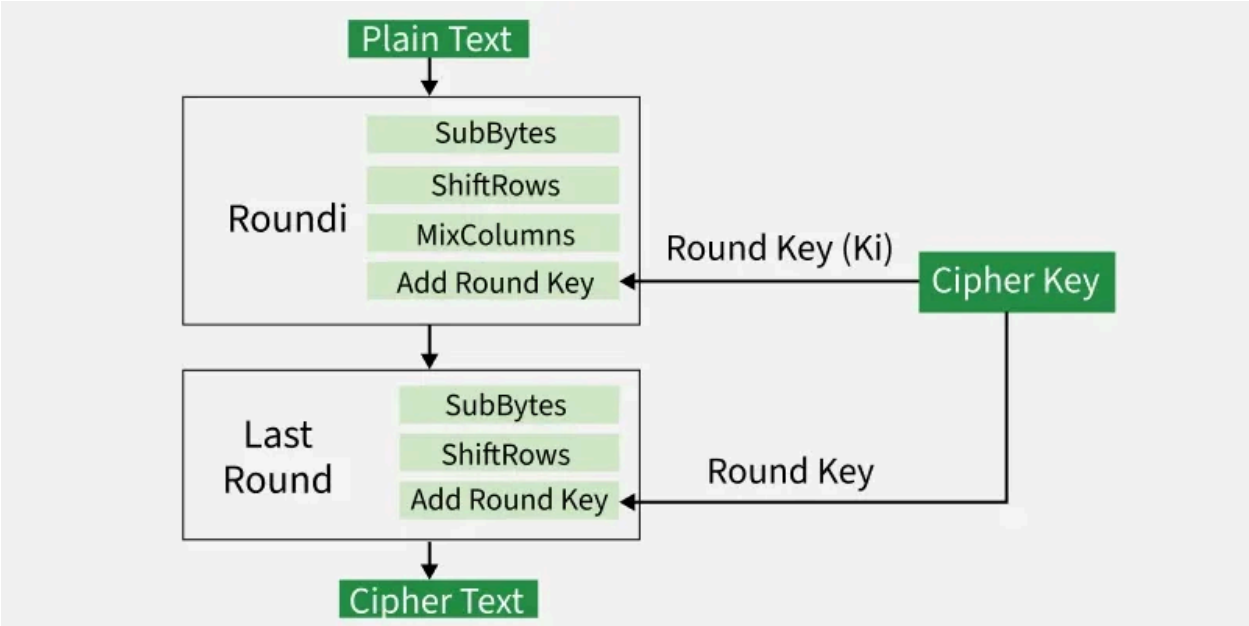
A Key Schedule algorithm calculates all the round keys from the key. So the initial key is used to create many different round keys which will be used in the corresponding round of the encryption.



Encryption

AES considers each block as a 16-byte (4 byte x 4 byte = 128) grid in a column-major arrangement.

[b0 | b4 | b8 | b12 |
| b1 | b5 | b9 | b13 |
| b2 | b6 | b10| b14 |
| b3 | b7 | b11| b15]



Each round comprises of 4 steps :

- SubBytes
- ShiftRows
- MixColumns
- Add Round Key

Step1. Sub Bytes

This step implements the substitution.

In this step, each byte is substituted by another byte. It is performed using a lookup table also called the S-box. This substitution is done in a way that a

byte is never substituted by itself and also not substituted by another byte which is a compliment of the current byte. The result of this step is a 16-byte (4 x 4) matrix like before.

The next two steps implement the permutation.

Step2. Shift Rows

This step is just as it sounds. Each row is shifted a particular number of times.

- The first row is not shifted
- The second row is shifted once to the left.
- The third row is shifted twice to the left.
- The fourth row is shifted thrice to the left.

(A left circular shift is performed.)

```
[ b0  | b1  | b2  | b3  ]        [ b0  | b1  | b2  | b3  ]
| b4  | b5  | b6  | b7  |   ->   | b5  | b6  | b7  | b4  |
| b8  | b9  | b10 | b11 |        | b10 | b11 | b8  | b9  |
[ b12 | b13 | b14 | b15 ]        [ b15 | b12 | b13 | b14 ]
```

Step 3: Mix Columns

This step is a matrix multiplication. Each column is multiplied with a specific matrix and thus the position of each byte in the column is changed as a result.

This step is skipped in the last round.

```
[ c0 ]      [ 2  3  1  1 ] [ b0 ]
| c1 | =    | 1  2  3  1 |   | b1 |
```

```
| c2 |     | 1  1  2  3 |     | b2 |
[ c3 ]     [ 3  1  1  2 ]     [ b3 ]
```

Step 4: Add Round Keys

- Now the resultant output of the previous stage is XOR-ed with the corresponding round key. Here, the 16 bytes are not considered as a grid but just as 128 bits of data.
- After all these rounds 128 bits of encrypted data are given back as output. This process is repeated until all the data to be encrypted undergoes this process.

Decryption

The stages in the rounds can be easily undone as these stages have an opposite to it which when performed reverts the changes. Each 128 blocks goes through the 10,12 or 14 rounds depending on the key size.

The stages of each round of decryption are as follows :

- Add round key
- Inverse MixColumns
- ShiftRows
- Inverse SubByte

The decryption process is the encryption process done in reverse so I will explain the steps with notable differences.

Inverse MixColumns

- This step is similar to the Mix Columns step in encryption but differs in the matrix used to carry out the operation.
- Mix Columns Operation each column is mixed independent of the other.
- Matrix multiplication is used. The output of this step is the matrix multiplication of the old values and a

constant matrix

[b0] = [14 11 13 9] [c0]
[b1]=[9 14 11 13] [c1]
[b2] =[13 9 14 11] [c2]
[b3]=[11 13 9 14] [c3]

Inverse SubBytes

- Inverse S-box is used as a lookup table and using which the bytes are substituted during decryption.
- Function Substitute performs a byte substitution on each byte of the input word. For this purpose, it uses an S-box.

Applications of AES

AES is widely used in many applications which require secure data storage and transmission. Some common use cases include:

- **Wireless security:** AES is used in securing wireless networks, such as Wi-Fi networks, to ensure data confidentiality and prevent unauthorized access.
- **Database Encryption:** AES can be applied to encrypt sensitive data stored in databases. This helps protect personal information, financial records, and other confidential data from unauthorized access in case of a data breach.
- **Secure communications:** AES is widely used in protocols such as internet communications, email, instant messaging, and voice/video calls. It ensures that the data remains confidential.
- **Data storage:** AES is used to encrypt sensitive data stored on hard drives, USB drives, and other storage media, protecting it from unauthorized access in case of loss or theft.
- **Virtual Private Networks (VPNs):** AES is commonly used in VPN protocols to secure the communication between a user's device and a remote server. It ensures that data sent and received through the VPN remains private and cannot be deciphered by eavesdroppers.
- **Secure Storage of Passwords:** AES encryption is commonly employed to store passwords securely. Instead of storing plaintext passwords, the encrypted version is stored. This adds an extra layer of security and protects user credentials in case of unauthorized access to the storage.

- **File and Disk Encryption:** AES is used to encrypt files and folders on computers, external storage devices, and cloud storage. It protects sensitive data stored on devices or during data transfer to prevent unauthorized access.

Stream Ciphers

In stream cipher, one byte is encrypted at a time while in block cipher ~128 bits are encrypted at a time. Initially, a key(k) will be supplied as input to pseudorandom bit generator and then it produces a random 8-bit output which is treated as keystream. The resulted keystream will be of size 1 byte, i.e., 8 bits. Stream ciphers are fast because they encrypt data bit by bit or byte by byte, which makes them efficient for encrypting large amounts of data quickly.Stream ciphers work well for real-time communication, such as video streaming or online gaming, because they can encrypt and decrypt data as it's being transmitted.

Key Points of Stream Cipher

1. Stream Cipher follows the sequence of pseudorandom number stream.
2. One of the benefits of following stream cipher is to make cryptanalysis more difficult, so the number of bits chosen in the Keystream must be long in order to make cryptanalysis more difficult.

3. By making the key more longer it is also safe against brute force attacks.
4. The longer the key the stronger security is achieved, preventing any attack.
5. Keystream can be designed more efficiently by including more number of 1s and 0s, for making cryptanalysis more difficult.
6. Considerable benefit of a stream cipher is, it requires few lines of code compared to block cipher.

Encryption

For Encryption,

- Plain Text and Keystream produces Cipher Text (Same keystream will be used for decryption.).
- The Plaintext will undergo XOR operation with keystream bit-by-bit and produces the Cipher Text.

Example:

Plain Text : 10011001

Keystream : 11000011

```
^^^^^^^^^^^^^^^^^^^^^^
```

Cipher Text : 01011010

Decryption

For Decryption,

- Cipher Text and Keystream gives the original Plain Text (Same keystream will be used for encryption.).
- The Ciphertext will undergo XOR operation with keystream bit-by-bit and produces the actual Plain Text.

Example:

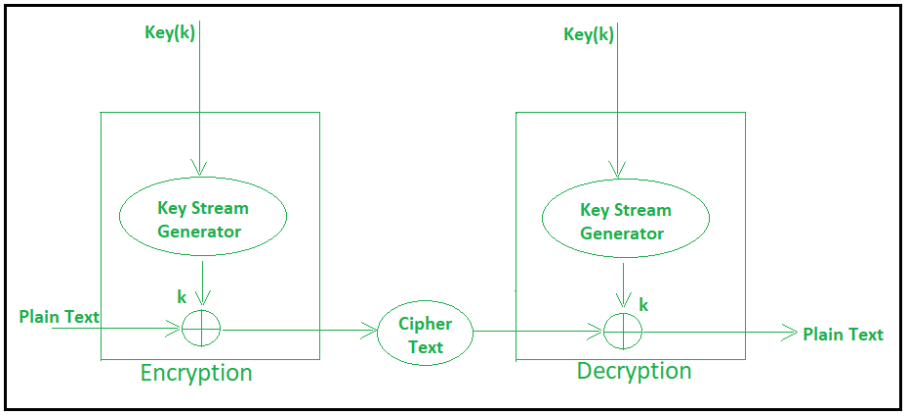
Cipher Text : 01011010

Keystream : 11000011

```
``````````````````````
```

*Plain Text  : 10011001*

Decryption is just the reverse process of Encryption i.e. performing XOR with Cipher Text.


```

Advantages of Stream Ciphers

Stream ciphers have many advantages, such as:

- **Speed:** Generally, this type of encryption is quicker than others, such as block ciphers.
- **Low complexity:** Stream ciphers are simple to implement into contemporary software, and developers don't require sophisticated hardware to do so.
- **Sequential in nature:** Certain companies handle communications written in a continuous manner. Stream ciphers enable them to transmit data when it's ready instead of waiting for everything to be finished because of their bit-by-bit processing.
- **Accessibility:** Using symmetrical encryption methods like stream ciphers saves businesses from having to deal with public and private keys. Additionally, computers are able to select the appropriate decryption key to utilize thanks to mathematical concepts behind current stream ciphers.

Disadvantages of Stream Ciphers

- If an error occurs during transmission, it can affect subsequent bits, potentially corrupting the entire message because stream ciphers rely on previously stored cipher bits for decryption
- Maintaining and properly distributing keys to stream ciphers can be difficult, especially in large systems or networks.

- Some stream ciphers may be predictable or vulnerable to attack if their key stream is not properly designed, potentially compromising the security of the encrypted data.

RC4

RC4 (Rivest Cipher 4) is a symmetric-key stream cipher designed by Ron Rivest in 1987 for RSA Security. Unlike block ciphers such as DES and AES, which encrypt data in fixed-size blocks, RC4 encrypts one byte (or one bit) at a time by generating a pseudo-random keystream. Each byte of plaintext is combined with a byte of the keystream using the Exclusive-OR (XOR) operation to produce the ciphertext.

RC4 gained popularity because of its simplicity, high speed, and low memory requirements. It was widely used in protocols such as SSL/TLS, WEP (Wired Equivalent Privacy), and WPA for wireless network security. However, researchers later discovered several weaknesses in RC4, and it is no longer recommended for modern cryptographic applications.

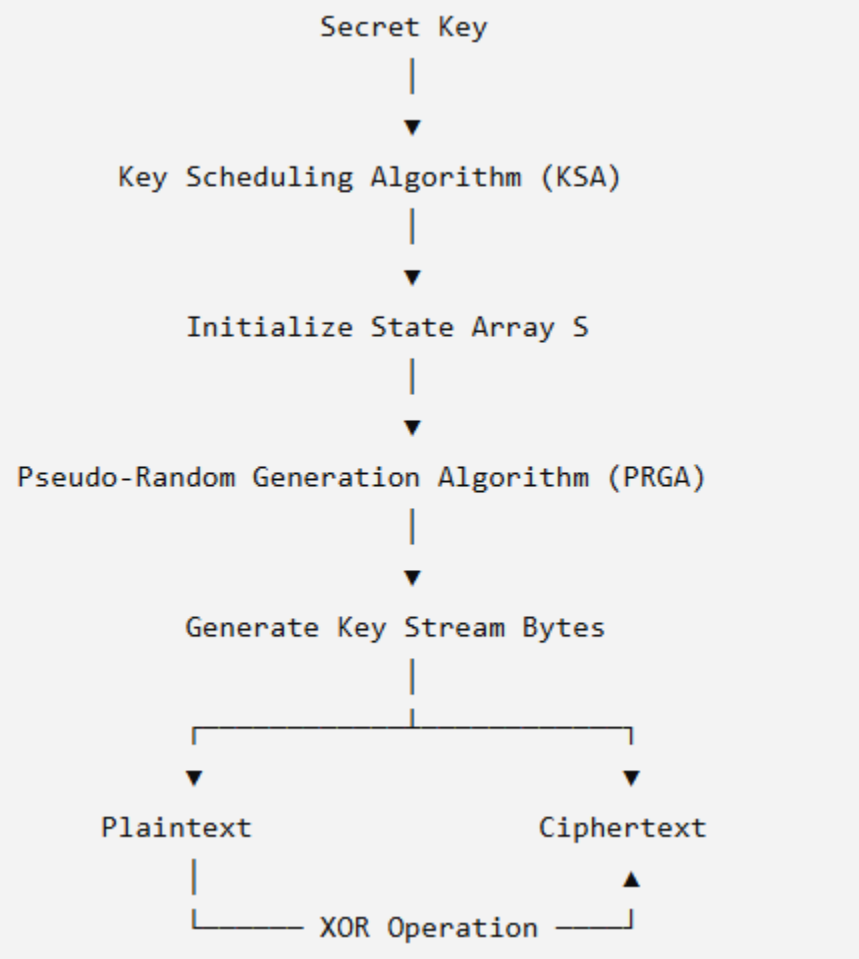
Basic Working of RC4

RC4 consists of **two major algorithms**:

1. **Key Scheduling Algorithm (KSA)**
2. **Pseudo-Random Generation Algorithm (PRGA)**

The KSA initializes the internal state using the secret key, while the PRGA generates the keystream used for encryption and decryption.

RC4 Architecture



RC4 Encryption Process

The RC4 encryption process consists of the following steps:

1. Enter the secret key.
2. Execute the Key Scheduling Algorithm (KSA).
3. Generate the keystream using PRGA.
4. XOR the plaintext with the keystream.
5. Obtain the ciphertext.

Step 1: Secret Key

The sender and receiver share a secret key.

Example:

Secret Key = COMPUTER

The key can have a length ranging from 40 bits to 256 bits.

Step 2: Key Scheduling Algorithm (KSA)

The Key Scheduling Algorithm (KSA) initializes the internal state array S.

Initially,

S[0] = 0

S[1] = 1

S[2] = 2

...

S[255] = 255

The state array contains **256 values**.

Working of KSA

Step 1

Initialize S[0...255]

Step 2

Repeat for i = 0 to 255

Step 3

Calculate

$$j = (j + S[i] + Key[i \bmod KeyLength]) \bmod 256$$

Step 4

Swap

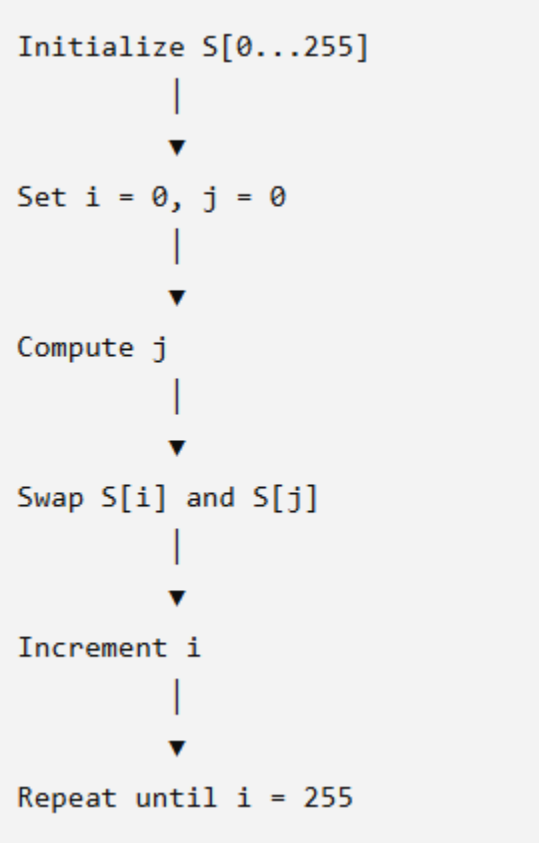
S[i]

↓

S[j]

The repeated swapping creates a random-looking permutation of the numbers 0–255.

KSA Flow Diagram



Purpose of KSA

The Key Scheduling Algorithm:

- Initializes the state array.
- Mixes the secret key with the state array.
- Produces a unique internal state.
- Ensures that different keys produce different keystreams.

Step 3: Pseudo-Random Generation Algorithm (PRGA)

After initialization, RC4 generates one keystream byte at a time.

Working of PRGA

For every plaintext byte:

Step 1: Increment

$$i=(i+1) \bmod 256$$

Step 2: Update

$$j = (j + S[i]) \bmod 256$$

Step 3: Swap

S[i]
↓
S[j]

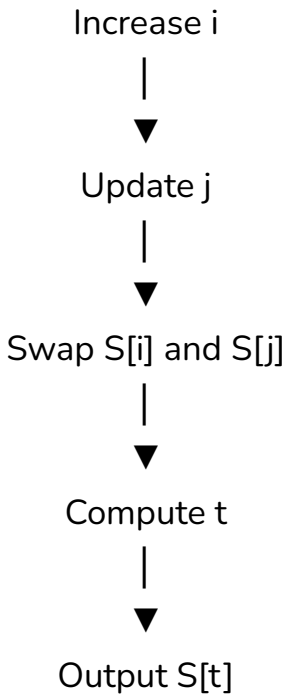
Step 4: Generate

$$t = (S[i] + S[j]) \bmod 256$$

Step 5: Output

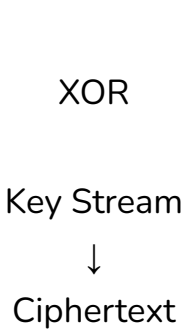
Key Stream Byte = S[t]

PRGA Flow Diagram



Step 4: Encryption

The generated keystream is XORed with the plaintext.



Mathematically,

$$\textbf{Ciphertext} = \textbf{Plaintext} \oplus \textbf{KeyStream}$$

Example

Plaintext
11001010
Key Stream
10101111
↓
Ciphertext
01100101

RC4 Decryption

RC4 uses the same operation for decryption.

Ciphertext
XOR
Same Key Stream
↓
Plaintext

Mathematically,

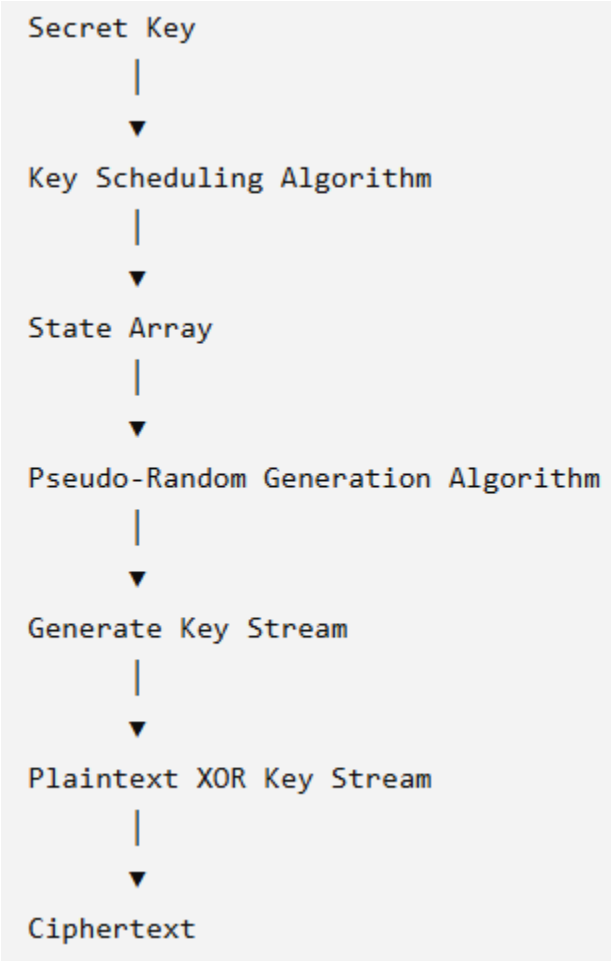
$$\textbf{Plaintext} = \textbf{Ciphertext} \oplus \textbf{KeyStream}$$

Since

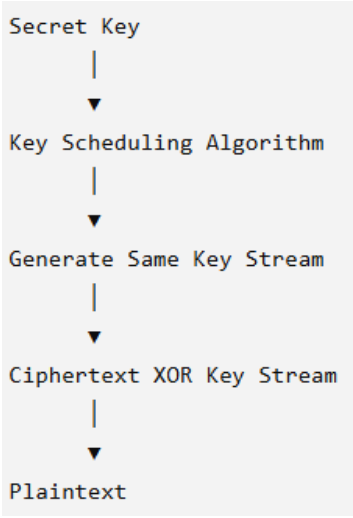
$$\mathbf{A} \oplus \mathbf{B} \oplus \mathbf{B} = \mathbf{A}$$

the original plaintext is recovered.

Complete RC4 Encryption Flow



RC4 Decryption Flow



Advantages of RC4

1. Simple Algorithm

RC4 is easy to understand and implement because it uses simple operations such as array indexing, swapping, and XOR.

2. High Speed

RC4 is very fast since it encrypts one byte at a time without requiring complex mathematical computations.

3. Low Memory Requirement

It uses only a 256-byte state array, making it suitable for devices with limited memory.

4. Variable Key Length

Supports secret keys ranging from 40 bits to 256 bits, providing flexibility for different applications.

5. Efficient for Streaming Data

RC4 is suitable for continuous data transmission such as audio, video, and network communication.

Limitations of RC4

1. Weak Initial Keystream

The first few bytes of the generated keystream exhibit statistical biases, allowing attackers to recover information about the key.

2. Key Reuse Vulnerability

Using the same key for multiple messages can reveal relationships between plaintexts and compromise security.

3. Statistical Attacks

RC4 is vulnerable to attacks that exploit predictable patterns in the keystream.

4. Deprecated Standard

Modern security protocols have removed RC4 because of its known weaknesses.

5. Not Suitable for Modern Applications

Protocols such as TLS no longer recommend RC4 for secure communication.

Applications of RC4

Although deprecated today, RC4 was historically used in:

- Secure Socket Layer (SSL)
- Transport Layer Security (TLS)
- Wired Equivalent Privacy (WEP)
- Wi-Fi Protected Access (WPA)
- PDF document encryption
- Remote desktop protocols

Public Key Encryption

Public key cryptography provides a secure way to exchange information and authenticate users by using pairs of keys. The public key is used for encryption and signature verification, while the private key is used for decryption and signing.

When the two parties communicate with each other to transfer the intelligible or sensible message, referred to as plaintext, is converted into apparently random unreadable for security purposes referred to as ciphertext.

Public key cryptography is a method of secure communication that uses a pair of keys, a public key, which anyone can use to encrypt messages or verify signatures, and a private key, which is kept secret and used to decrypt messages or sign documents.

This system ensures that only the intended recipient can read an encrypted message and that a signed message truly comes from the claimed sender. Public key cryptography is essential for secure internet communications, allowing for confidential messaging, authentication of identities, and verification of data integrity.

Cryptographic Key

A cryptographic key is a piece of information used by cryptographic algorithms to encrypt or decrypt data, authenticate identities, or generate digital signatures. It serves as a parameter to control cryptographic operations, ensuring the security and privacy of digital communications and transactions.

How Does TLS/SSL Use Public Key Cryptography

TLS/SSL uses public key cryptography to keep our internet connections secure. It does this in two main ways:

- **Encryption**: When you visit a secure website (HTTPS), TLS/SSL helps encrypt data exchanged between your browser and the website's server.
  It uses a combination of public and private keys to create a secure connection. Your browser and the server agree on a secret key for this session, which keeps your data safe from eavesdroppers.
- **Authentication**: TLS/SSL verifies the identity of websites. When you connect to a site, it presents a digital certificate signed by a trusted authority. Your browser checks this certificate to ensure you're really connecting to the right site and not a fake one trying to steal your information.

By using public key cryptography, TLS/SSL protects our privacy online and ensures that the websites we visit are genuine and trustworthy.

Components of Public Key Encryption

- **Plain Text:** This is the message which is readable or understandable. This message is given to the Encryption algorithm as an input.
- **Cipher Text:** The cipher text is produced as an output of Encryption algorithm. We cannot simply understand this message.
- **Encryption Algorithm:** The encryption algorithm is used to convert plain text into cipher text.
- **Decryption Algorithm:** It accepts the cipher text as input and the matching key (Private Key or Public key) and produces the original plain text.
- **Public and Private Key:** One key either Private key (Secret key) or Public Key (known to everyone) is used for encryption and other is used for decryption.

Public Key Encryption Working

Key Pair Generation : A user generates a pair of keys :

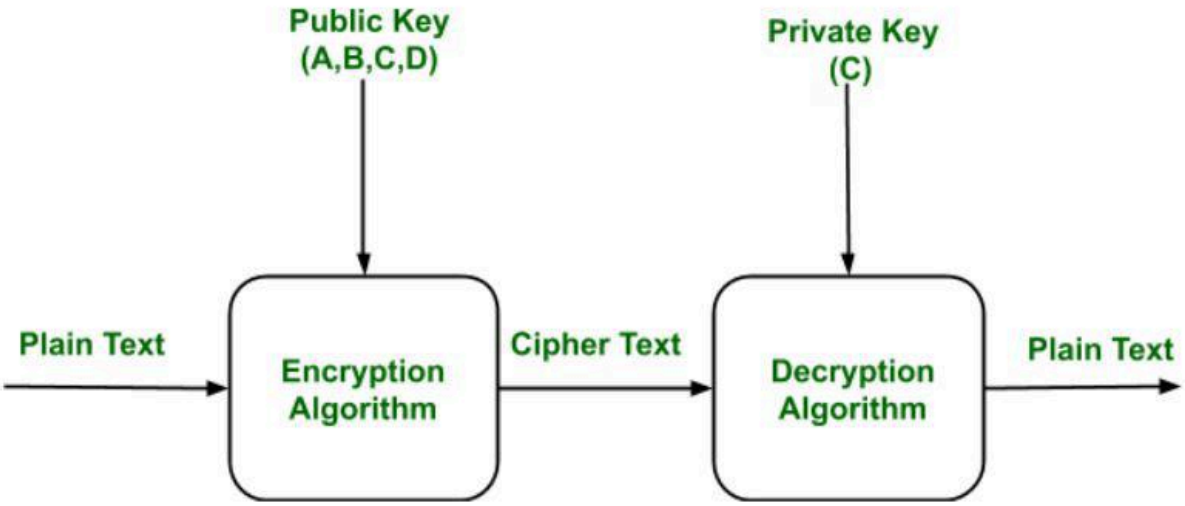
- **Public Key**: Shared openly. Anyone can use it to send an encrypted message.
- **Private Key**: Kept secret. Only the key owner can decrypt messages encrypted with the public key.

Encryption : If someone wants to send a private message:

- They obtain the recipient's public key.
- They encrypt the message using that public key.
- The encrypted message is sent over a network.

Decryption : Upon receiving the message:

- The recipient uses their private key to decrypt the message and recover the original plaintext



Public Key Encryption Practical Example: **Secure Website (HTTPS)**

When you visit a secure website like https://www.bank.com, public key encryption is used behind the scenes to encrypt data between your browser and the bank's server.

Bank's Server Has a Key Pair

- **Private Key**: Secret, stored securely on the server.
- **Public Key**: Shared with anyone via an SSL certificate.

You Connect to the Website

- Your browser gets the bank's public key from its SSL certificate.
- It verifies the certificate is valid (issued by a trusted certificate authority).

Encrypting the Session Key

- Your browser creates a random symmetric key (used for actual data encryption).
- It encrypts this key using the bank's public key.
- Only the bank can decrypt it using its private key.

Secure Communication Begins

- Now both your browser and the bank share a secret symmetric key.
- All further communication (login info, account data, etc.) is encrypted using this key.

Why Public Key Encryption is Used

- It ensures that only the server (with the private key) can read the symmetric key.

- Even if someone intercepts the traffic, they can't decrypt the session key or data.

Characteristics of Public Encryption key

- **Security Assurance**:

  It is computationally infeasible to determine the private (decryption) key from the public (encryption) key and algorithm alone.
- **Key Pair Flexibility**:

  Either key (public or private) can be used for encryption, with the other used for decryption supporting both confidentiality and authentication.
- **Easy Public Key Distribution**:

  Public keys can be shared freely, enabling convenient encryption and digital signature verification.
- **Private Key Confidentiality**:

  Private keys are kept secret, ensuring that only the key owner can decrypt content or create valid digital signatures.
- **Foundation of RSA**:
  The most widely used public-key cryptosystem, RSA, is based on the difficulty of factoring large composite numbers into primes.

Limitations of the Public Key Encryption

- **Susceptible to Brute-Force Attacks**: Although computationally hard, public key encryption algorithms can be theoretically brute

forced if key lengths are too short or computational power advances (e.g., quantum computing).

- **Private Key Loss**: If a user loses their private key, they can no longer decrypt data or prove their identity, making the system non-recoverable and highly vulnerable.
- **Man-in-the-Middle (MitM) Attack Risk**: A third party could intercept and alter public keys during transmission, leading to unauthorized decryption or spoofed signatures if keys aren't verified through a trusted channel.
- **PKI Chain of Trust Vulnerability**: If a private key higher in the PKI hierarchy (e.g., a root certificate authority) is compromised, it can invalidate all subordinate certificates, enabling widespread MitM attacks.

Applications of the Public Key Encryption

- **SSL/TLS protocols** : They use public key encryption to securely exchange symmetric session keys between a web browser and a server.
- **Digital signature:** Digital signature is for senders authentication purpose. In this sender encrypt the plain text using his own private key. This step will make sure the authentication of the sender because receiver can decrypt the cipher text using senders public key only.
- **Key exchange:** This algorithm can use in both Key-management and securely transmission of data.

- **SSH keys** : For secure login to remote servers use public/private key pairs for authentication.
- **Blockchain and Cryptocurrencies**: Users control wallets with private keys , public keys serve as wallet addresses.

Requirements for Public Key Cryptosystem

A **public key cryptosystem** (asymmetric cryptography) uses two mathematically related keys — a **public key** for encryption or signature verification and a **private key** for decryption or signing. To ensure its security and functionality, certain requirements must be met.

Core requirements:

1. Easy Key Pair Generation – It must be computationally simple to generate a valid public-private key pair based on the chosen mathematical function (e.g., RSA uses large prime factorization) .
2. Efficient Encryption – Given the public key and encryption algorithm, producing ciphertext from plaintext should be computationally easy for legitimate users .
3. Efficient Decryption – The intended recipient should be able to decrypt ciphertext quickly using their private key .
4. One-Way Security – Knowing the public key must not allow an attacker to feasibly determine the corresponding private key .
5. Ciphertext Security – Even with access to ciphertext and the public key, it should be computationally infeasible for an attacker to recover the original plaintext .

6. Key Pair Reversibility – The system should support both confidentiality and authentication by allowing encryption with one key and decryption with the other: D[PU, E(PR, M)] = D[PR, E(PU, M)] .

Security Characteristics:

- Computational Infeasibility – Breaking the system should require impractical computational resources (e.g., factoring large numbers in RSA) .
- Public Key Distribution – Public keys can be shared openly without compromising security, but must be verified to prevent man-in-the-middle attacks .
- Private Key Confidentiality – The private key must remain secret; loss or compromise renders the system insecure .

Example – In TLS/SSL, the server's public key encrypts a session key, and only the server's private key can decrypt it, ensuring secure communication .

Best Practices:

- Use sufficiently large key sizes (e.g., 2048-bit RSA or higher) to resist brute-force attacks.
- Employ trusted certificate authorities for public key verification.
- Regularly rotate keys to limit exposure in case of compromise.

These requirements ensure that a public key cryptosystem provides confidentiality, authentication, integrity, and non-repudiation in secure communications.